

# Encapsulamiento

---

Clase 3

# Encapsulamiento

El encapsulamiento o encapsulación hace referencia al ocultamiento de los estados internos de un objeto al exterior. Dicho de otra manera, encapsular consiste en hacer que los atributos o métodos internos a una clase no se puedan acceder ni modificar desde fuera, sino que tan solo el propio objeto pueda acceder a ellos.

Pero

“Las **variables o métodos «privados»** de instancia, que no pueden accederse excepto desde dentro de un objeto, **no existen en Python**”

# Atributos y métodos “no públicos” en Python

Al no existir encapsulamiento en Python, se ha acordado una convención entre los programadores. La cual consiste en anteponer el caracter `_` antes del nombre del atributo o método para informar que no se debería acceder a esta desde afuera de la clase. Aunque en la realidad el acceso aun sigue siendo posible.

La funcionalidad del doble guión bajo (name mangling) `__` es para que no haya conflictos de nombres con subclasses aunque también crea un falso encapsulamiento como el simple guión bajo.

# Ejemplo

```
class Test:
    def __init__(self, n1):
        self.n1 = n1
        self._n2 = 22 # privado
        self.__n3 = 33 # name mangling

    def get_n2(self):
        return self._n2

    def get_n3(self):
        return self.__n3

    def _metodo_privado(self):
        print("Este es un método privado")

    def __metodo_privado2(self):
        print("Este es un método privado 2")
```

```
t = Test(11)
# print(dir(t))

print(t.n1)
# print(t.n2)
# print(t.n3)
# print(t._n2)      # No hacer esto
# print(t.__n3)     # No hacer esto
# print(t._Test__n3) # No hacer esto
# t.metodo_privado()
# t._metodo_privado() # No hacer esto
```

# Getters y Setters

```
class Persona:
    def __init__(self, nombre: str):
        self._nombre = nombre

    def get_nombre(self):
        return self._nombre

    def set_nombre(self, nombre):
        self._nombre = nombre

if __name__ == '__main__':
    p1 = Persona('Juan')
    print(p1.get_nombre())
    p1.set_nombre('Jose')
    print(p1.get_nombre())
```

```
class Persona:
    def __init__(self, nombre: str):
        self._nombre = nombre

    @property          # decorator setter
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, nuevo_nombre):
        # Aquí puedes agregar validaciones.
        self._nombre = nuevo_nombre

if __name__ == '__main__':
    p1 = Persona('Juan')
    print(p1.nombre)
    p1.nombre = 'Jose'
    print(p1.nombre)
```

# Ejercicio 1

Crear una clase jugador que tenga como atributos un nombre, email, password y teléfono. Un objeto de esta clase solo podrá cambiar la password y el teléfono si ingresa la contraseña correcta (con la que fue instanciada) sino se deberá levantar una excepción.

## Ejercicio 2

Crear una clase MensajeSecreto la cual tenga como atributos un mensaje y una clave. Además debe tener un método que devuelva el mensaje secreto si es que la clave recibida como parámetro es la correcta. Si no es así, deberá retornar un mensaje de error.

## Ejercicio 3

Un centro de educación (nombre, dirección, email) requiere de un programa en el cual se pueda dar de alta a docentes (dni, nombre), alumnos (dni, nombre), y materias (nombre, año, dni del docente, listado de dni alumnos, precio).

El centro de educación deberá poder:

Informar cuánto recauda cada materia.

Informar cuantas materias dicta el docente con dni '98989898'



## Ejercicio 4

Un hotel (nombre, dirección, teléfono) requiere un programa para dar de alta y baja a los huéspedes (dni, nombre y apellido, cant\_noches, número de habitación).

Cuenta con 5 habitaciones (ocupada?, precio, número). Los cobros se realizarán al realizar el check-out, se deberá desocupar la habitación e informar cuanto se le cobró al huésped.

Para el check-in se deberá comprobar si hay habitaciones disponibles y marcar como ocupada la habitación designada.